How Workers for Platforms works

Copy Page

Workers for Platforms is built on top of Cloudflare Workers. The same security and performance models used by Workers apply to applications that use Workers for Platforms.

The Workers configuration API was initially built around managing a relatively small number of Workers on each account. This leads to some difficulties when using Workers as a platform for your own users, including:

- Frequently needing to increase script limits.
- Adding an ever-increasing number of routes.
- Managing logic in a central place if your own logic is supposed to come before your customers' logic.

Workers for Platforms extends the capabilities of Workers for SaaS businesses that want to deploy Worker scripts on behalf of their customers or that want to let their users write Worker scripts directly.

Architecture

Workers for Platforms introduces a new architecture model as outlined on this page.

Dispatch namespace

A dispatch namespace is composed of a collection of user Workers. With dispatch namespaces, a dynamic dispatch Worker can be used to call any user Worker in a namespace.

Best practice

Having a production and staging namespace is useful to test changes that you have made to your dynamic dispatch Worker.

If you have multiple distinct services you are providing your customers, you should split these out into different dispatch workers and namespaces. We discourage creating a new namespace for each customer.

Dynamic dispatch Worker

A dynamic dispatch Worker is written by Cloudflare's platform customers to run their own logic before dispatching (routing) the request to user Workers. In addition to routing, it can be used to run authentication, create boilerplate functions and sanitize responses.

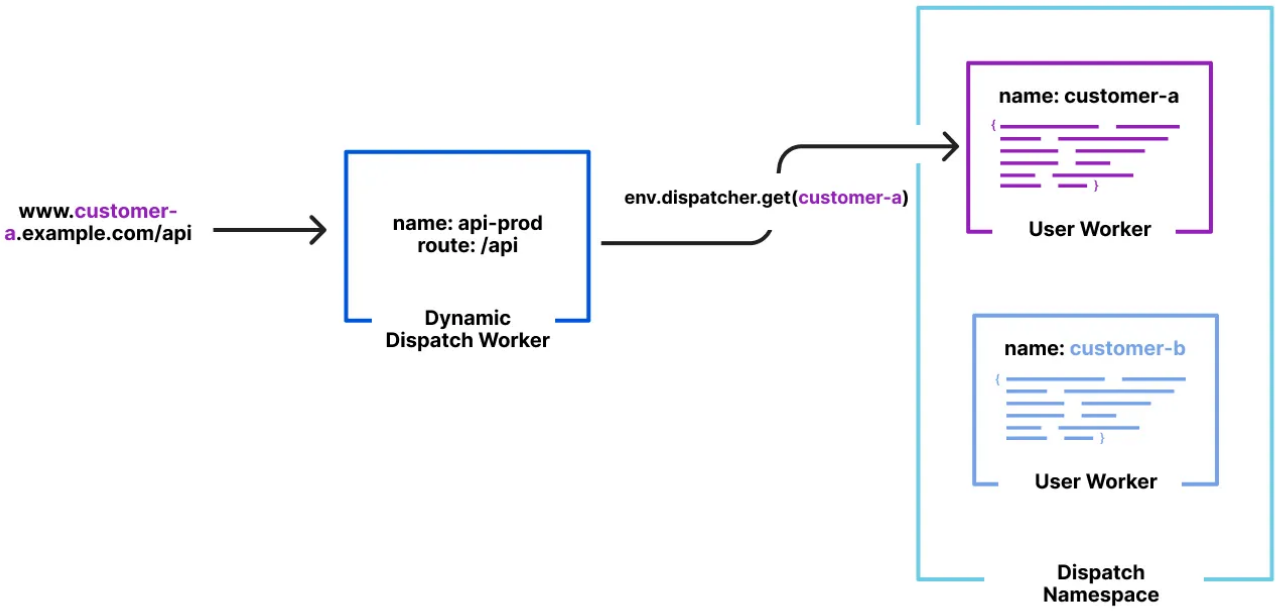
The dynamic dispatch Worker calls user Workers from the dispatch namespace and executes them. The dynamic dispatch Worker is configured with a dispatch namespace binding. The binding is the entrypoint for all requests to user Workers.

User Workers

User Workers are written by your end users (end developers). End developers deploy user Workers to script automated actions, create integrations or modify response payloads to return custom content.

Request lifecycle

Below you will find an example request lifecycle in the Workers for Platforms architecture.



In the above diagram:

1. Request for `customer-a.example.com/api` will first hit the dynamic dispatch Worker (`api-prod`).
2. The dispatcher (`env.dispatcher.get(customer-a)`) configured in your dynamic dispatch Worker code will handle routing logic to user Workers.
3. The subdomain (`customer-a.example.com`) of the incoming request is used to route to the user Worker with the same name (`customer-a`).

Workers for Platforms versus Service bindings

Both Workers for Platforms and Service bindings enable Worker-to-Worker communication.

Service bindings explicitly link two Workers together. They are meant for use cases where you know exactly which Workers need to communicate with each other. Service bindings do not work in the Workers for Platforms model because user Workers are uploaded as needed by your end users.

In the Workers for Platforms model, a dynamic dispatch Worker can be used to call any user Worker (similar to how Service bindings work) in a dispatch namespace but without needing to

explicitly pre-define the relationship.

Service bindings and Workers for Platforms can be used simultaneously when building applications.

Cache API

Workers for Platforms user Workers have access to namespaced cache through the cache API. Namespaced cache is isolated across user Workers. For isolation, `caches.default` is disabled for namespaced scripts.

To learn more about the cache, refer to How the cache Works.

Was this helpful?

Previous
← Reference

Next
User Worker metadata ↗ →

Last updated: Feb 25, 2025

Company

cloudflare.com

Our team

Careers

Tools

Cloudflare Radar

Speed Test

Is BGP Safe Yet?

RPKI Toolkit

Certificate Transparency

Community

X

Discord

YouTube

GitHub